

冲击前端架构师——工程化思维与编译原理详解

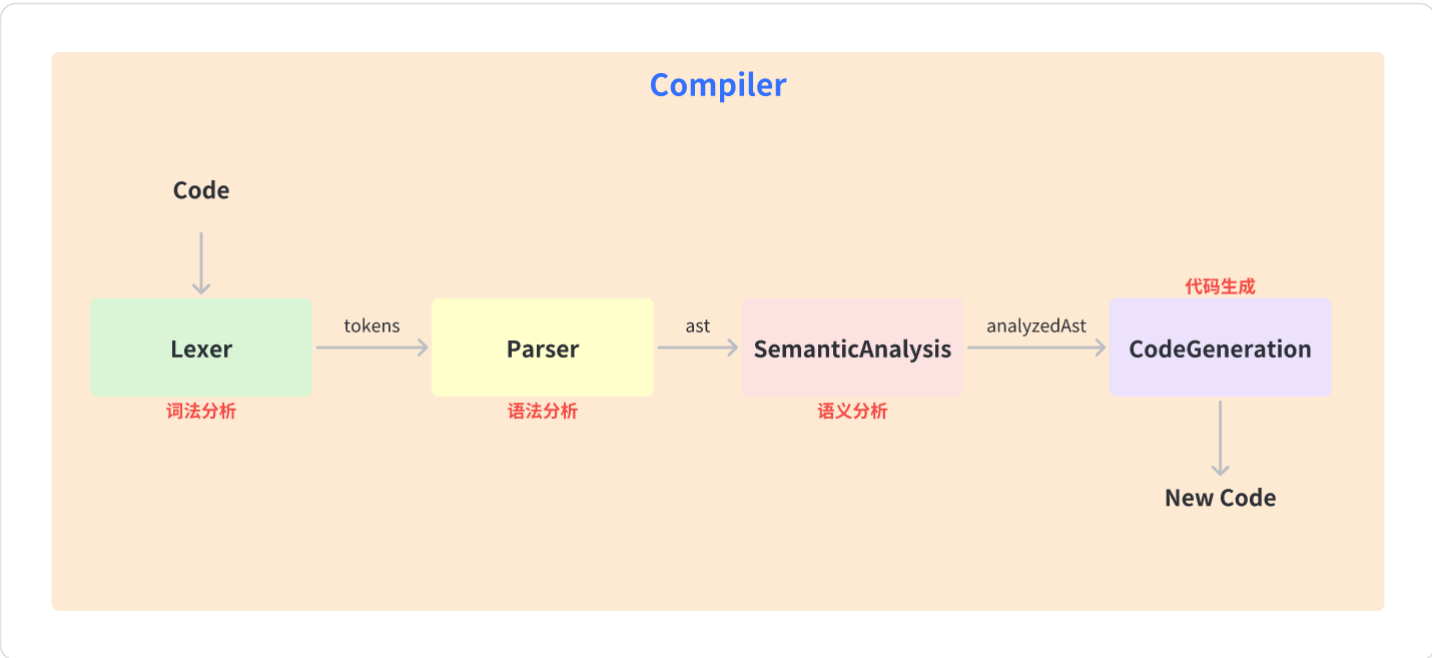
大家备好茶水，这节课会很干。

面试真题1：了解过 AST 吗，请说说它的运用场景

代码的本质——字符串

字符串的一些操作，就是所谓的编译

主谓宾



-  1. 词法分析（Lexical Analysis）：将源代码转换成单词流，称为“词法单元”（tokens），每个词法单元包含一个标识符和一个属性值，比如变量名、数字、操作符等等。
- 2. 语法分析（Parsing）：将词法单元流转换成抽象语法树（Abstract Syntax Tree，简称AST），也就是标记所构成的数据结构，表示源代码的结构和规则。
- 3. 语义分析（Semantic Analysis）：在AST上执行类型检查、作用域检查等操作，以确保代码的正确性和安全性。
- 4. 代码生成（Code Generation）：基于AST生成目标代码，包括优化代码结构、生成代码文本、进行代码压缩等等。

下面是一个简单的JavaScript编译器示例代码：

其中：

`lexer` 是词法分析器，将源代码转换成词法单元流；

`parser` 是语法分析器，将词法单元流转换成抽象语法树；

`semanticAnalysis` 是语义分析器，对抽象语法树进行语义分析；

`codeGeneration` 是代码生成器，将分析后的AST生成目标代码。

一个编译器最核心的代码

```
1 function compiler(sourceCode) {
2   // 词法分析
3   const tokens = lexer(sourceCode);
4
5   // 语法分析
6   const ast = parser(tokens);
7
8   // 语义分析
9   const analyzedAst = semanticAnalysis(ast);
10
11  // 代码生成
12  const code = codeGeneration(analyzedAst);
13
14  return code;
15 }
```

为什么在工作中需要用到编译原理，一个公式编辑器、一个字符串复杂处理。低代码平台更是需要详细掌握 AST 及编译原理。airtable、coda、glide、fibery

	LISP	C
1		
2		
3	2 + 2	add(2, 2)
4	4 - 2	subtract(4, 2)
5	2 + (4 - 2)	add(2, subtract(4, 2))

将 LISP 语言的代码转为 C

编译器示例

```
1 function tokenizer(input) {
2
3   // A `current` variable for tracking our position in the code like a cursor.
```

```
4  let current = 0;
5
6  // And a `tokens` array for pushing our tokens to.
7  let tokens = [];
8
9  // We start by creating a `while` loop where we are setting up our `current`
10 // variable to be incremented as much as we want `inside` the loop.
11 //
12 // We do this because we may want to increment `current` many times within a
13 // single loop because our tokens can be any length.
14 while (current < input.length) {
15
16   // We're also going to store the `current` character in the `input`.
17   let char = input[current];
18
19   // The first thing we want to check for is an open parenthesis. This will
20   // later be used for `CallExpression` but for now we only care about the
21   // character.
22   //
23   // We check to see if we have an open parenthesis:
24   if (char === '(') {
25
26     // If we do, we push a new token with the type `paren` and set the value
27     // to an open parenthesis.
28     tokens.push({
29       type: 'paren',
30       value: '(',
31     });
32
33     // Then we increment `current`
34     current++;
35
36     // And we `continue` onto the next cycle of the loop.
37     continue;
38   }
39
40   // Next we're going to check for a closing parenthesis. We do the same exact
41   // thing as before: Check for a closing parenthesis, add a new token,
42   // increment `current`, and `continue`.
43   if (char === ')') {
44     tokens.push({
45       type: 'paren',
46       value: ')',
47     });
48     current++;
49     continue;
50   }
```

```
51
52 // Moving on, we're now going to check for whitespace. This is interesting
53 // because we care that whitespace exists to separate characters, but it
54 // isn't actually important for us to store as a token. We would only throw
55 // it out later.
56 //
57 // So here we're just going to test for existence and if it does exist we're
58 // going to just `continue` on.
59 let WHITESPACE = /\s/;
60 if (WHITESPACE.test(char)) {
61     current++;
62     continue;
63 }
64
65 // The next type of token is a number. This is different than what we have
66 // seen before because a number could be any number of characters and we
67 // want to capture the entire sequence of characters as one token.
68 //
69 // (add 123 456)
70 //      ^^^  ^^^
71 //      Only two separate tokens
72 //
73 // So we start this off when we encounter the first number in a sequence.
74 let NUMBERS = /[0-9]/;
75 if (NUMBERS.test(char)) {
76
77     // We're going to create a `value` string that we are going to push
78     // characters to.
79     let value = '';
80
81     // Then we're going to loop through each character in the sequence until
82     // we encounter a character that is not a number, pushing each character
83     // that is a number to our `value` and incrementing `current` as we go.
84     while (NUMBERS.test(char)) {
85         value += char;
86         char = input[++current];
87     }
88
89     // After that we push our `number` token to the `tokens` array.
90     tokens.push({ type: 'number', value });
91
92     // And we continue on.
93     continue;
94 }
95
96 // We'll also add support for strings in our language which will be any
97 // text surrounded by double quotes (").
```

```

98 //
99 // (concat "foo" "bar")
100 //      ^^^  ^^^ string tokens
101 //
102 // We'll start by checking for the opening quote:
103 if (char === '"') {
104     // Keep a `value` variable for building up our string token.
105     let value = '';
106
107     // We'll skip the opening double quote in our token.
108     char = input[++current];
109
110     // Then we'll iterate through each character until we reach another
111     // double quote.
112     while (char !== '"') {
113         value += char;
114         char = input[++current];
115     }
116
117     // Skip the closing double quote.
118     char = input[++current];
119
120     // And add our `string` token to the `tokens` array.
121     tokens.push({ type: 'string', value });
122
123     continue;
124 }
125
126 // The last type of token will be a `name` token. This is a sequence of
127 // letters instead of numbers, that are the names of functions in our lisp
128 // syntax.
129 //
130 // (add 2 4)
131 //      ^^^
132 //      Name token
133 //
134 let LETTERS = /[a-z]/i;
135 if (LETTERS.test(char)) {
136     let value = '';
137
138     // Again we're just going to loop through all the letters pushing them to
139     // a value.
140     while (LETTERS.test(char)) {
141         value += char;
142         char = input[++current];
143     }
144

```

```

145     // And pushing that value as a token with the type `name` and continuing.
146     tokens.push({ type: 'name', value });
147
148     continue;
149 }
150
151     // Finally if we have not matched a character by now, we're going to throw
152     // an error and completely exit.
153     throw new TypeError('I dont know what this character is: ' + char);
154 }
155
156     // Then at the end of our `tokenizer` we simply return the tokens array.
157     return tokens;
158 }
159
160 /**
161  * =====
162  *                               \/*o   o\/*
163  *                               THE PARSER!!!
164  * =====
165  */
166
167 /**
168  * For our parser we're going to take our array of tokens and turn it into an
169  * AST.
170  *
171  * [{ type: 'paren', value: '(' }, ...] => { type: 'Program', body: [...]}
172  */
173
174 // Okay, so we define a `parser` function that accepts our array of `tokens`.
175 function parser(tokens) {
176
177     // Again we keep a `current` variable that we will use as a cursor.
178     let current = 0;
179
180     // But this time we're going to use recursion instead of a `while` loop. So we
181     // define a `walk` function.
182     function walk() {
183
184         // Inside the walk function we start by grabbing the `current` token.
185         let token = tokens[current];
186
187         // We're going to split each type of token off into a different code path,
188         // starting off with `number` tokens.
189         //
190         // We test to see if we have a `number` token.
191         if (token.type === 'number') {

```

```
192
193     // If we have one, we'll increment `current`.
194     current++;
195
196     // And we'll return a new AST node called `NumberLiteral` and setting its
197     // value to the value of our token.
198     return {
199         type: 'NumberLiteral',
200         value: token.value,
201     };
202 }
203
204 // If we have a string we will do the same as number and create a
205 // `StringLiteral` node.
206 if (token.type === 'string') {
207     current++;
208
209     return {
210         type: 'StringLiteral',
211         value: token.value,
212     };
213 }
214
215 // Next we're going to look for CallExpressions. We start this off when we
216 // encounter an open parenthesis.
217 if (
218     token.type === 'paren' &&
219     token.value === '('
220 ) {
221
222     // We'll increment `current` to skip the parenthesis since we don't care
223     // about it in our AST.
224     token = tokens[++current];
225
226     // We create a base node with the type `CallExpression`, and we're going
227     // to set the name as the current token's value since the next token after
228     // the open parenthesis is the name of the function.
229     let node = {
230         type: 'CallExpression',
231         name: token.value,
232         params: [],
233     };
234
235     // We increment `current` *again* to skip the name token.
236     token = tokens[++current];
237
238     // And now we want to loop through each token that will be the `params` of
```

```

239 // our `CallExpression` until we encounter a closing parenthesis.
240 //
241 // Now this is where recursion comes in. Instead of trying to parse a
242 // potentially infinitely nested set of nodes we're going to rely on
243 // recursion to resolve things.
244 //
245 // To explain this, let's take our Lisp code. You can see that the
246 // parameters of the `add` are a number and a nested `CallExpression` that
247 // includes its own numbers.
248 //
249 // (add 2 (subtract 4 2))
250 //
251 // You'll also notice that in our tokens array we have multiple closing
252 // parenthesis.
253 //
254 // [
255 //   { type: 'paren', value: '(' },
256 //   { type: 'name', value: 'add' },
257 //   { type: 'number', value: '2' },
258 //   { type: 'paren', value: '(' },
259 //   { type: 'name', value: 'subtract' },
260 //   { type: 'number', value: '4' },
261 //   { type: 'number', value: '2' },
262 //   { type: 'paren', value: ')' }, <<< Closing parenthesis
263 //   { type: 'paren', value: ')' }, <<< Closing parenthesis
264 // ]
265 //
266 // We're going to rely on the nested `walk` function to increment our
267 // `current` variable past any nested `CallExpression`.
268
269 // So we create a `while` loop that will continue until it encounters a
270 // token with a `type` of `paren` and a `value` of a closing
271 // parenthesis.
272 while (
273   (token.type !== 'paren') ||
274   (token.type === 'paren' && token.value !== ')')
275 ) {
276   // we'll call the `walk` function which will return a `node` and we'll
277   // push it into our `node.params`.
278   node.params.push(walk());
279   token = tokens[current];
280 }
281
282 // Finally we will increment `current` one last time to skip the closing
283 // parenthesis.
284 current++;
285

```



```

286     // And return the node.
287     return node;
288 }
289
290 // Again, if we haven't recognized the token type by now we're going to
291 // throw an error.
292 throw new TypeError(token.type);
293 }
294
295 // Now, we're going to create our AST which will have a root which is a
296 // `Program` node.
297 let ast = {
298     type: 'Program',
299     body: [],
300 };
301
302 // And we're going to kickstart our `walk` function, pushing nodes to our
303 // `ast.body` array.
304 //
305 // The reason we are doing this inside a loop is because our program can have
306 // `CallExpression` after one another instead of being nested.
307 //
308 // (add 2 2)
309 // (subtract 4 2)
310 //
311 while (current < tokens.length) {
312     ast.body.push(walk());
313 }
314
315 // At the end of our parser we'll return the AST.
316 return ast;
317 }
318
319 /**
320  * =====
321  *                               ^(*>  <*)^
322  *                               THE TRAVERSER!!!
323  * =====
324  */
325
326 /**
327  * So now we have our AST, and we want to be able to visit different nodes with
328  * a visitor. We need to be able to call the methods on the visitor whenever we
329  * encounter a node with a matching type.
330  *
331  * traverse(ast, {
332  *     Program: {

```

```

333 *     enter(node, parent) {
334 *         // ...
335 *     },
336 *     exit(node, parent) {
337 *         // ...
338 *     },
339 * },
340 *
341 *     CallExpression: {
342 *         enter(node, parent) {
343 *             // ...
344 *         },
345 *         exit(node, parent) {
346 *             // ...
347 *         },
348 *     },
349 *
350 *     NumberLiteral: {
351 *         enter(node, parent) {
352 *             // ...
353 *         },
354 *         exit(node, parent) {
355 *             // ...
356 *         },
357 *     },
358 * });
359 */
360
361 // So we define a traverser function which accepts an AST and a
362 // visitor. Inside we're going to define two functions...
363 function traverser(ast, visitor) {
364
365     // A `traverseArray` function that will allow us to iterate over an array and
366     // call the next function that we will define: `traverseNode`.
367     function traverseArray(array, parent) {
368         array.forEach(child => {
369             traverseNode(child, parent);
370         });
371     }
372
373     // `traverseNode` will accept a `node` and its `parent` node. So that it can
374     // pass both to our visitor methods.
375     function traverseNode(node, parent) {
376
377         // We start by testing for the existence of a method on the visitor with a
378         // matching `type`.
379         let methods = visitor[node.type];

```

```

380
381 // If there is an `enter` method for this node type we'll call it with the
382 // `node` and its `parent`.
383 if (methods && methods.enter) {
384     methods.enter(node, parent);
385 }
386
387 // Next we are going to split things up by the current node type.
388 switch (node.type) {
389
390     // We'll start with our top level `Program`. Since Program nodes have a
391     // property named body that has an array of nodes, we will call
392     // `traverseArray` to traverse down into them.
393     //
394     // (Remember that `traverseArray` will in turn call `traverseNode` so we
395     // are causing the tree to be traversed recursively)
396     case 'Program':
397         traverseArray(node.body, node);
398         break;
399
400     // Next we do the same with `CallExpression` and traverse their `params`.
401     case 'CallExpression':
402         traverseArray(node.params, node);
403         break;
404
405     // In the cases of `NumberLiteral` and `StringLiteral` we don't have any
406     // child nodes to visit, so we'll just break.
407     case 'NumberLiteral':
408     case 'StringLiteral':
409         break;
410
411     // And again, if we haven't recognized the node type then we'll throw an
412     // error.
413     default:
414         throw new TypeError(node.type);
415 }
416
417 // If there is an `exit` method for this node type we'll call it with the
418 // `node` and its `parent`.
419 if (methods && methods.exit) {
420     methods.exit(node, parent);
421 }
422 }
423
424 // Finally we kickstart the traverser by calling `traverseNode` with our ast
425 // with no `parent` because the top level of the AST doesn't have a parent.
426 traverseNode(ast, null);

```

```

427 }
428
429 /**
430 * =====
431 *          (          )
432 *          THE TRANSFORMER!!!
433 * =====
434 */
435
436 /**
437  * Next up, the transformer. Our transformer is going to take the AST that we
438  * have built and pass it to our traverser function with a visitor and will
439  * create a new ast.
440  *
441  * -----
442  *   Original AST           |   Transformed AST
443  * -----
444  *   {                       |   {
445  *     type: 'Program',      |     type: 'Program',
446  *     body: [{              |     body: [{
447  *       type: 'CallExpression', |     type: 'ExpressionStatement',
448  *       name: 'add',          |     expression: {
449  *       params: [{           |       type: 'CallExpression',
450  *         type: 'NumberLiteral', |       callee: {
451  *         value: '2'            |         type: 'Identifier',
452  *       }, {                  |         name: 'add'
453  *         type: 'CallExpression', |       },
454  *         name: 'subtract',      |       arguments: [{
455  *         params: [{           |         type: 'NumberLiteral',
456  *           type: 'NumberLiteral', |         value: '2'
457  *           value: '4'            |       }, {
458  *         }, {                  |         type: 'CallExpression',
459  *           type: 'NumberLiteral', |         callee: {
460  *           value: '2'            |           type: 'Identifier',
461  *         }]                   |           name: 'subtract'
462  *       }, {                  |       },
463  *       type: 'CallExpression', |       arguments: [{
464  *       name: 'subtract',      |         type: 'NumberLiteral',
465  *       params: [{           |         value: '4'
466  *         type: 'NumberLiteral', |       }, {
467  *         value: '2'            |         type: 'NumberLiteral',
468  *       }, {                  |         value: '2'
469  *       type: 'CallExpression', |       }]
470  *       name: 'subtract',      |     }
471  *       params: [{           |   }
472  *         type: 'NumberLiteral', |   }]
473  *         value: '2'            | }

```

```

474  * -----
475  */
476
477  // So we have our transformer function which will accept the lisp ast.
478  function transformer(ast) {
479
480      // We'll create a `newAst` which like our previous AST will have a program
481      // node.
482      let newAst = {
483          type: 'Program',
484          body: [],
485      };
486
487      // Next I'm going to cheat a little and create a bit of a hack. We're going to
488      // use a property named `context` on our parent nodes that we're going to push
489      // nodes to their parent's `context`. Normally you would have a better
490      // abstraction than this, but for our purposes this keeps things simple.
491      //
492      // Just take note that the context is a reference *from* the old ast *to* the
493      // new ast.
494      ast._context = newAst.body;
495
496      // We'll start by calling the traverser function with our ast and a visitor.
497      traverser(ast, {
498
499          // The first visitor method accepts any `NumberLiteral`
500          NumberLiteral: {
501              // We'll visit them on enter.
502              enter(node, parent) {
503                  // We'll create a new node also named `NumberLiteral` that we will push
504                  // the parent context.
505                  parent._context.push({
506                      type: 'NumberLiteral',
507                      value: node.value,
508                  });
509              },
510          },
511
512          // Next we have `StringLiteral`
513          StringLiteral: {
514              enter(node, parent) {
515                  parent._context.push({
516                      type: 'StringLiteral',
517                      value: node.value,
518                  });
519              },
520          },

```

```

521
522 // Next up, `CallExpression`.
523 CallExpression: {
524     enter(node, parent) {
525
526         // We start creating a new node `CallExpression` with a nested
527         // `Identifier`.
528         let expression = {
529             type: 'CallExpression',
530             callee: {
531                 type: 'Identifier',
532                 name: node.name,
533             },
534             arguments: [],
535         };
536
537         // Next we're going to define a new context on the original
538         // `CallExpression` node that will reference the `expression`'s argument
539         // so that we can push arguments.
540         node._context = expression.arguments;
541
542         // Then we're going to check if the parent node is a `CallExpression`.
543         // If it is not...
544         if (parent.type !== 'CallExpression') {
545
546             // We're going to wrap our `CallExpression` node with an
547             // `ExpressionStatement`. We do this because the top level
548             // `CallExpression` in JavaScript are actually statements.
549             expression = {
550                 type: 'ExpressionStatement',
551                 expression: expression,
552             };
553         }
554
555         // Last, we push our (possibly wrapped) `CallExpression` to the `parent`
556         // `context`.
557         parent._context.push(expression);
558     },
559 }
560 });
561
562 // At the end of our transformer function we'll return the new ast that we
563 // just created.
564 return newAst;
565 }
566
567 /**

```

```

568 * =====
569 *                                     \"/ (// ^ \n ^) /
570 *                                     THE CODE GENERATOR!!!!
571 * =====
572 */
573
574 /**
575  * Now let's move onto our last phase: The Code Generator.
576  *
577  * Our code generator is going to recursively call itself to print each node in
578  * the tree into one giant string.
579  */
580
581 function codeGenerator(node) {
582
583     // We'll break things down by the `type` of the `node`.
584     switch (node.type) {
585
586         // If we have a `Program` node. We will map through each node in the `body`
587         // and run them through the code generator and join them with a newline.
588         case 'Program':
589             return node.body.map(codeGenerator)
590                 .join('\n');
591
592         // For `ExpressionStatement` we'll call the code generator on the nested
593         // expression and we'll add a semicolon...
594         case 'ExpressionStatement':
595             return (
596                 codeGenerator(node.expression) +
597                 ';' // << (...because we like to code the *correct* way)
598             );
599
600         // For `CallExpression` we will print the `callee`, add an open
601         // parenthesis, we'll map through each node in the `arguments` array and run
602         // them through the code generator, joining them with a comma, and then
603         // we'll add a closing parenthesis.
604         case 'CallExpression':
605             return (
606                 codeGenerator(node.callee) +
607                 '(' +
608                 node.arguments.map(codeGenerator)
609                     .join(', ') +
610                 ')'
611             );
612
613         // For `Identifier` we'll just return the `node`'s name.
614         case 'Identifier':

```

```

615     return node.name;
616
617     // For `NumberLiteral` we'll just return the `node`'s value.
618     case 'NumberLiteral':
619         return node.value;
620
621     // For `StringLiteral` we'll add quotations around the `node`'s value.
622     case 'StringLiteral':
623         return '"' + node.value + '"';
624
625     // And if we haven't recognized the node, we'll throw an error.
626     default:
627         throw new TypeError(node.type);
628 }
629 }
630
631 /**
632  * =====
633  *                                     ( * ‘ワ’) ”
634  *                               !!!!!!!!!THE COMPILER!!!!!!!!!
635  * =====
636  */
637
638 /**
639  * FINALLY! We'll create our `compiler` function. Here we will link together
640  * every part of the pipeline.
641  *
642  * 1. input => tokenizer    => tokens
643  * 2. tokens => parser      => ast
644  * 3. ast   => transformer => newAst
645  * 4. newAst => generator  => output
646  */
647
648 function compiler(input) {
649     let tokens = tokenizer(input);
650     let ast    = parser(tokens);
651     let newAst = transformer(ast);
652     let output = codeGenerator(newAst);
653
654     // and simply return the output!
655     return output;
656 }
657
658 /**
659  * =====
660  *                                     (๑    )
661  * !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!YOU MADE IT!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

```



```

662  * =====
663  */
664
665  // Now I'm just exporting everything...
666  module.exports = {
667    tokenizer,
668    parser,
669    traverser,
670    transformer,
671    codeGenerator,
672    compiler,
673  };

```

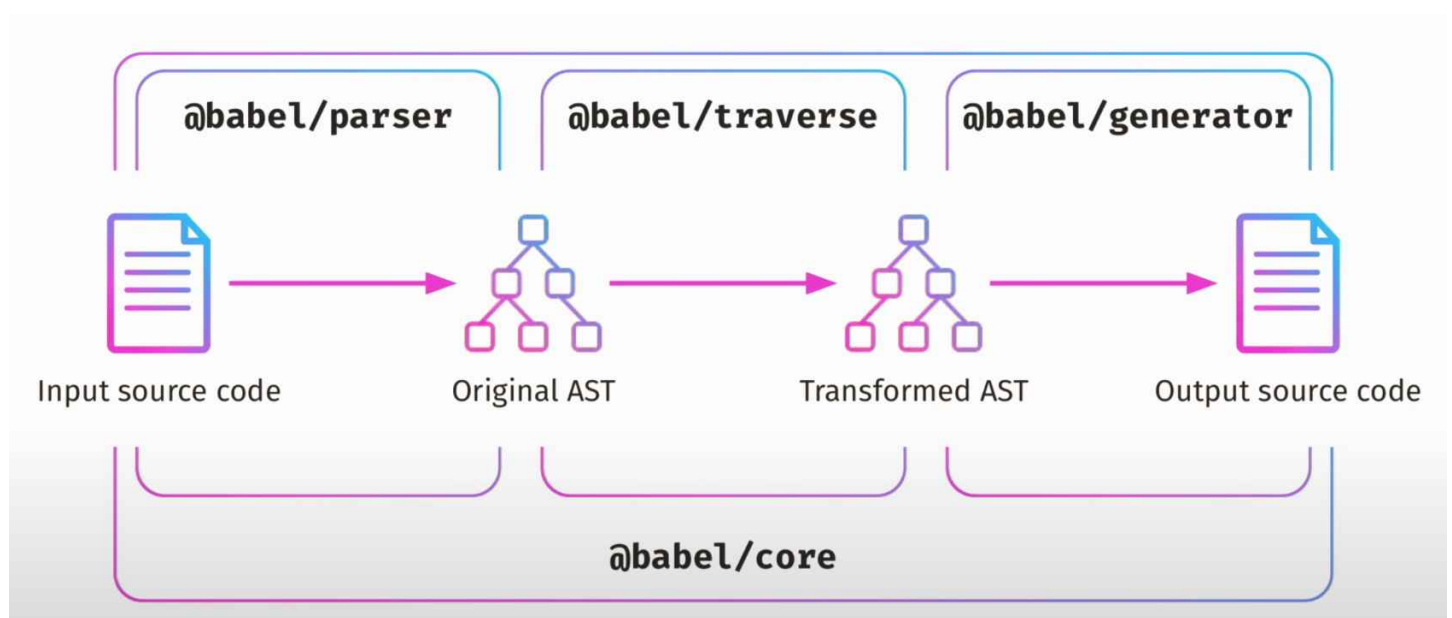
面试真题2: babel 的 plugin 和 loader 应用与原理

babel 是一个流行的 JavaScript 编译器

babel 包含以下几个核心内容:

1. @babel/core
2. @babel/parser
3. @babel/traverse
4. @babel/generator
5. 辅助相关, type polyfill temple 等

babel 的预设 babel-preset-env



```

1 `
2 import %%importName%% from "%%source%%"
3 `
4 期望这段代码最终输出:
5 `
6 import module from 'module'
7 `
8
9 // console.log(code)
10 // import module from 'module'
11
12 import template from "@babel/template";
13 import generate from "@babel/generator";
14 import * as t from "@babel/types";
15
16 const buildRequire = template(`
17   var %%importName%% = require(%%source%%);
18 `);
19
20 const ast = buildRequire({
21   importName: t.identifier("module"),
22   source: t.stringLiteral("module"),
23 });
24
25 console.log(generate(ast).code);
26

```

开发并调试 babel 插件

开始之前我们先给大家介绍一下我们经常使用的链式调用语法

1. 创建项目：首先，创建一个新的npm项目，并安装 `@babel/core`、`@babel/parser`、`@babel/traverse`、`@babel/types` 依赖项。

```

1 $ mkdir custom-babel-plugin
2 $ cd custom-babel-plugin
3 $ npm init -y
4 $ npm install --save-dev @babel/core @babel/parser @babel/traverse @babel/types

```

1. 创建插件文件：在项目目录下创建一个新的JavaScript文件，例如 `custom-plugin.js`。
2. 编写插件代码：在 `custom-plugin.js` 文件中编写自定义插件代码。Babel插件是一个函数，接受一个 `babel` 参数，可以使用这个参数访问Babel提供的API。

```

1 module.exports = function customPlugin(babel) {
2   const { types: t } = babel; // 获取 Babel types API
3
4   return {
5     name: "custom-plugin", // 插件名称
6     visitor: { // 访问者对象，用于访问抽象语法树节点
7       CallExpression(path, state) {
8         // 如果调用的函数名是 "customFunc", 则在函数调用前添加一条日志
9         if (path.node.callee.name === "customFunc") {
10           const logStatement = t.StringLiteral("Calling customFunc...");
11           path.insertAfter(t.ExpressionStatement(logStatement));
12         }
13       }
14     }
15   };
16 };

```

以上插件代码定义了一个名为 `customPlugin` 的插件函数，它将在Babel编译中被调用。插件代码使用访问者模式遍历抽象语法树，查找符合条件的语法结构，并进行转换。示例代码展示了一个针对 `CallExpression` 节点的访问者对象，如果节点调用名是 `customFunc`，则在节点后面插入一条日志语句。

1. 配置Babel：在项目根目录创建 `.babelrc` 文件，告诉Babel要使用自定义插件。

```

1 {
2   "plugins": ["../custom-plugin.js"]
3 }

```

在以上配置中，将自定义插件文件路径添加到了 `plugins` 数组中。

1. 测试插件：编写一些测试代码用来测试自定义插件。
2. 运行测试：使用 `@babel/cli` 命令行工具来编译JavaScript代码，并输出到控制台。

```

1 $ npx babel test.js

```

1. 调试插件：使用 `@babel/parser` 将JavaScript代码解析成抽象语法树，然后将语法树作为参数传递给插件以进行调试。

```

1 const parser = require("@babel/parser");
2 const traverse = require("@babel/traverse");

```

```
3 const generate = require("@babel/generator");
4
5 const code = "customFunc()";
6 const ast = parser.parse(code);
7 traverse.default(ast, customPlugin()); // 调用自定义插件进行转换
8 const output = generate.default(ast);
9
10 console.log(output.code); // 输出转换后的代码
```

以上调试代码将JavaScript代码解析成抽象语法树，然后将语法树作为参数传递给自定义插件，最终输出转换后的代码。

babel 除了转译 code 以外，还可以做什么呢？

polyfill 等

面试真题3：请说说 webpack 打包过程与原理

Webpack 基本的配置掌握情况如何？

1. splitChunk 怎么做
2. Tree shaking
3. Dll
4. Css 提取
5. Terser 压缩
6. mode、entry、output、module (loader) 、resolve、external、plugin

Webpack 构建流程

几个核心概念：

1. Compiler
2. Compilation
3. Module
4. Chunk
5. Bundle

执行过程描述

1. 初始化，初始化会读取配置信息、统计入口文件、解析 loader 及 plugin 等信息；
2. 编译阶段，webpack 编译代码，部分依赖 babel，ts 转为 JavaScript，less 转为 css，styled-components 进行处理
3. 输出阶段：生成输出文件，包含文件名，输出路径，资源信息

初始化阶段的主要流程

1. 初始化参数
2. 创建 compiler 对象实例
3. 开始编译，compiler.run
4. 确定入口，根据 entry，找出所有入口文件，调用 addEntry

构建阶段

1. 编译模块，通过 entry 对应的 dependence 创建 module 对象，调用对应 loader 去将模块转为 js 内容，babel 将一些内容转换为目标内容
2. 完成模块编译，得到一个 moduleGraph

生成阶段

1. 输出资源组装 chunk，chunkGroup，再将 Chunk 转换为一个单独文件加入到输出列表，既然到这儿已经加入到输出列表了，说明这里是修改资源内容的最后机会也就是（afterChunks: new SyncHook(["chunks"])) 钩子）
2. 写入文件系统（emitAssets）在确定好输出内容后，根据配置输出到文件中

插件

- `compiler.hooks.compilation` :
 - 时机：启动编译创建出 compilation 对象后触发
 - 参数：当前编译的 compilation 对象
 - 示例：很多插件基于此事件获取 compilation 实例
- `compiler.hooks.make` :
 - 时机：正式开始编译时触发
 - 参数：同样是当前编译的 `compilation` 对象

- 示例：webpack 内置的 `EntryPlugin` 基于此钩子实现 `entry` 模块的初始化
- `compilation.hooks.optimizeChunks` :
 - 时机： `seal` 函数中， `chunk` 集合构建完毕后触发
 - 参数： `chunks` 集合与 `chunkGroups` 集合
 - 示例： `SplitChunksPlugin` 插件基于此钩子实现 `chunk` 拆分优化
- `compiler.hooks.done` :
 - 时机：编译完成后触发
 - 参数： `stats` 对象，包含编译过程中的各类统计信息
 - 示例： `webpack-bundle-analyzer` 插件基于此钩子实现打包分析

Plugin 的本质是对象

```
1 export default class CusPlugin {
2   constructor(options = {}) {
3     this.options = options;
4   }
5   apply(compiler) {
6     /*...*/
7   }
8 }
```

Loader 的本质是函数

```
1 const loaderUtils = require('loader-utils');
2
3 exports = module.exports = function(source) {
4   // 对 source 进行一些处理后...
5   return source;
6 };
```

```
1 const loaderUtils = require('loader-utils');
2
3 exports = module.exports = function(source) {
4   // 对 source 进行一些处理后...
5   return source;
6 };
```

自定义 Plugin

定义

我们前面提到过，Webpack 插件的本质是类，并且这个类必须定义 `apply` 方法，基于这些原则，我们首先定义一个最简单的 webpack 插件。实例代码如下：

```
1 export default class CusPlugin {
2   constructor(options = {}) {
3     this.options = options;
4   }
5   apply(compiler) {
6     /*...*/
7   }
8 }
```

通过以上示例，我们可以发现，自定义插件的核心逻辑在 `apply` 方法中执行，我们可以为已经定义的 hook 添加监听事件，从而在对应事件调用时，完成我们定义的操作。有了这个概念，我们接下来通过一个很常见的例子，深入了解自定义插件的定义与使用。现在有一个需求，需要在 webpack 打包完成后，将本次所有打包文件名称输出到 `fileList.md` 文件中。以上需求，我们提炼关键字，如下：

1. 打包完成时机
2. 打包生成资源
3. 将处理后的信息输出到 `fileList.md` 文件

针对于第一部分，打包完成时机，我们可以通过 `compiler` 对象上的 `hooks` 获取到 `emit` 钩子，然后为该钩子绑定一个新的事件函数。通过该钩子能够获取到 `compilation` 对象，通过该对象就能获取打包生成的资源。最终以 `fileList.md` 为名，为 `compilation` 指定新资源，从而实现 `fileList` 文件输出。

完善我们的 webpack plugin，代码示例如下：

```
1 export default class FileListPlugin {
2   constructor(options = {}) {
3     this.options = options;
4     this.filename = this.options.filename || 'fileList.md';
5   }
6   apply(compiler) {
7     // 打包完成时机
8     compiler.hooks.emit.tap('FileListPlugin', compilation => {
9       const { filename: fileName } = this;
```

```

10     const { assets } = compilation;
11     const fileCount = assets.length;
12     let content = `# 本次打包共生成${fileCount}个文件\n\n`;
13     // 遍历打包生成的资源
14     for (let filename in asstes) {
15         content += `- ${filename}\n`;
16     }
17     // 将信息输出到 fileList.md 文件并生成该文件
18     compilation.assets[fileListName] = {
19         source: function() {
20             return content;
21         },
22         size: function() {
23             return content.length;
24         },
25     };
26 });
27 }
28 }
29
30 exports = module.exports = FileListPlugin;

```

使用

在 webpack 中使用该插件：

```

1 // webpack.config.js
2
3 const path = require('path');
4 const FileListPlugin = require('./path/to/plugins/file-list-plugin');
5
6 module.exports = {
7     entry: './src/index.js',
8     output: {
9         filename: '[name].bundle.js',
10        path: path.resolve(__dirname, 'dist'),
11    },
12    plugins: [new FileListPlugin()],
13 };

```


自定义 Loader

定义

自定义 Loader 的本质是一个函数，该函数接收源码 source 参数，在这里首先需要明确一点，代码也不过是字符串，处理代码内容其实也就是字符串的处理，我们首先书写一个最简单的 loader，代码示例如下：

```
1 const loaderUtils = require('loader-utils');
2
3 exports = module.exports = function(source) {
4   // 对 source 进行一些处理后...
5   return source;
6 };
```

以上例子是一个最简单的 webpack loader，加入我们现在有一个需求，需要将给定代码中的模板内容替换为给定值。我们约定，将 “{{author}}” 替换为 “合一”。假设有一个文件，代码如下：

```
1 console.log('{{author}}欢迎你!');
```

接下来我们改进一下我们的 loader，示例如下：

```
1 // temp-loader.js
2
3 const loaderUtils = require('loader-utils');
4 const path = require('path');
5 const authorName = '合一';
6
7 exports = module.exports = function(source) {
8   const matches = source.match(/\{\{author\}\}/g);
9   for (const match of matches) {
10     source = source.replace(match, authorName);
11   }
12   return source;
13 };
```

使用

自定义 loader 需要在 webpack.config.js 中进行配置，配置不复杂，我们直接撈出代码：

```
1 // webpack.config.js
2
3 const path = require('path');
4
5 module.exports = {
6   target: 'node',
7   entry: './index',
8   output: {
9     path: path.resolve(__dirname, 'build'),
10    filename: '[name].js',
11  },
12  resolveLoader: {
13    modules: ['./node_modules', './loaders'],
14  },
15  module: {
16    rules: [
17      {
18        test: /\.js$/,
19        use: [
20          {
21            loader: 'temp-loader',
22          },
23        ],
24      },
25    ],
26  },
27 };
```

这样，在项目下执行 `yarn start`，或 `npx webpack` 就可以输出处理后的文件，文件内容为：

```
1 console.log('合一欢迎你!');
```

由此可见，哪怕是复杂 loader 的定义，也是对输入的源码 source 字符串进行处理，而后生成新的内容返回。

如果是 webpack-dev-server 呢？

Module -> chunk -> bundle